

COS341 2026 : Semester-Practical : Syntax Specification of the Students' Programming Language SPL [Version 2026]

General strategy: To avoid the awkward *lexing-ambiguities* which we have discussed in Chapter 1 (and Homework 1), we stipulate (require) that *every token must end with a blank space*. This will help the compiler to recognize clearly where one token ends before the next token begins.

For technical clarification,

blank space could possibly be:

- ASCII **code 32** (decimal) // the large elongated key at the bottom of the keyboard
 - ASCII **code 13** (decimal) // the 'new line' or 'return' or 'enter' key of the keyboard
-

Lexical category NUM

is characterised by the following **regular expression**, with epsilon ϵ meaning "nothing":

$0 |$
 $(-\epsilon)0.(0|1|2|3|4|5|6|7|8|9)^*(1|2|3|4|5|6|7|8|9) |$
 $(-\epsilon)(1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)^*.(0|1|2|3|4|5|6|7|8|9)^*(1|2|3|4|5|6|7|8|9) |$
 $(-\epsilon)(1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)^* |$

Lexical category USER-DEFINED-NAME

is characterised by the following **regular expression**, which is purposefully designed in such a way that there *cannot occur any tokenisation-conflict* between user-defined names and the reserved keywords of the language SPL:

$\#(0|1|2|3|4|5|6|7|8|9|a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z)^* |$

Lexical category STRING

is characterised by the following **regular expression**:

$"(,|.|:-?!|0|1|2|3|4|5|6|7|8|9|a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z)^*" |$

SPL's Context-Free-Grammar

by Prof.G.

keywords/tokens are **highlighted**

SPL_PROG	→	P\$ // comment: This is the special "Rule 0" for the End-of-File (\$)
P	→	V_DECL : F_DECL : ALGO
V_DECL	→	ϵ // nullable
V_DECL	→	<u>USER-DEFINED-NAME</u> V_DECL
F_DECL	→	ϵ // nullable
F_DECL	→	F_TYPE F_DECL
F_TYPE	→	void <u>USER-DEFINED-NAME</u> (V_DECL) { P return }
F_TYPE	→	num <u>USER-DEFINED-NAME</u> (V_DECL) { P return (TERM) }
ALGO	→	ϵ // nullable
ALGO	→	INSTR ; ALGO
OUTP	→	(TERM)
OUTP	→	<u>STRING</u>
INSTR	→	print OUTP
INSTR	→	nop // no-operation : could be used for empty else-cases (after if-then-)
INSTR	→	comment <u>STRING</u>
INSTR	→	ASSIGN
INSTR	→	BRANCH
INSTR	→	LOOP
INSTR	→	CALL
CALL	→	<u>USER-DEFINED-NAME</u> (INPUT)
INPUT	→	ϵ // nullable
INPUT	→	TERM INPUT
ASSIGN	→	<u>USER-DEFINED-NAME</u> = TERM
TERM	→	<u>USER-DEFINED-NAME</u>
TERM	→	<u>NUM</u>
TERM	→	CALL
TERM	→	mod (TERM TERM)
TERM	→	add (TERM TERM)
TERM	→	sub (TERM TERM)
TERM	→	mul (TERM TERM)
TERM	→	div (TERM TERM)
TERM	→	neg (TERM)
BRANCH	→	if BOOL then { ALGO } else { ALGO }
BOOL	→	not (BOOL)
BOOL	→	and (BOOL BOOL)
BOOL	→	or (BOOL BOOL)
BOOL	→	eq (TERM TERM)
BOOL	→	larger (TERM TERM)
BOOL	→	lesser (TERM TERM)
LOOP	→	COND BOOL do { ALGO }
LOOP	→	do { ALGO } COND BOOL
COND	→	while
COND	→	until

To do:

- Software Architecture: Together with all your project group members, discuss and decide whether you want to construct a lexer-less parser, or whether you want to "plug" a separate lexer module in front of the parser module.
- *Analyse whether the given SPL-Grammar is **suitable for LL(1)-Parsing**? If "yes", then construct an LL(1) Parser for it – otherwise discuss and decide with your project group members what to do and how to proceed...*

INPUT

On **Test-Day** [exact date to be announced] you will be *given* a plain ASCII text file **SPL.txt** for input, which your Parser (with or without Lexer) must analyse:

- *If* the given **SPL.txt** file does **not** contain a syntactically correct SPL Program, then your Parser must "throw" a meaningful **Syntax Error** Message (plus some additional helpful "hints" which the user can well understand).
- *Otherwise*, your Parser must **create and write out a structured XML file** with the name **tree.xml** that contains the given SPL program's *syntax tree* in the following form:

The OUTPUT file

The output file **tree.xml** contains all nodes of the syntax tree as XML structs –opened by suitable "tags" <...> and closed by suitable "tags" </...> (as it is usual in HTML as well as in XML)– which can be "rendered" by a web-browser such that the tutors can easily inspect the generated results via a web-browser.

Thereby,

- **The root node** is an **XML-struct** which contains the following **components**:
 - a new unique node ID number *which occurs nowhere else* in the entire syntax tree,
 - a "contents" field which contains the non-terminal Start-Symbol,
 - a "children" field which contains a list of numbers: these are the unique ID numbers of the immediate children nodes which are directly connected to this root.
- **An inner node** is an **XML-struct** which contains the following **components**:
 - a new unique node ID number *which occurs nowhere else* in the entire syntax tree,
 - a "contents" field which contains the relevant non-terminal from the SPL grammar,
 - a "children" field which contains a list of numbers: these are the unique ID numbers of the immediate children nodes which are directly connected to this node,
 - a "parent" field which contains the unique ID number of the "upper" node to which this given node is a child.
- **A leaf node** is an **XML struct** which contains the following **components**:
 - a new unique node ID number *which occurs nowhere else* in the entire syntax tree,
 - a "contents" field which contains the relevant **terminal token** *from the input stream* which the parser had previously "eaten": *If* you had decided to construct a lexer-less parser then this token is exactly *one* symbol – *otherwise*, however, the token can be a longer *sequence of symbols* which are treated as one lexical unit (for example a name, or a number).
 - a "parent" field which contains the unique ID number of the "upper" node to which this given leaf-node is a child.

Hint: The Unique ID Numbers will become important *later*, when every tree-node gets "linked" to a semantic data table that will contain additional information for name-scope-analysis, type-checking, etc... In other words: The data base (to be developed later) will use those unique node ID numbers as "foreign keys" according to the usual Database Theory. Thus, in later phases of the project your *parser's output file **tree.xml*** will serve as *input for the back-end modules* which will ultimately generate executable low-level target code from the given high-level SPL program.

You are allowed to use **Parser-Development-Tools**, which you *may perhaps* find on the internet, in support of the given project task. The tutors will **check only** the output **tree.xml**, not your Parser's software program

And now: HAPPY PROGRAMMING :)